Lab 3A: MCP - Connect Claude to Your Systems

Duration: 35 min

After: MCP Deep Dive lecture (Day 3, Block 1)

Prerequisites: Day 2 complete (toolkit plugin installed)

Objective

Configure an MCP server for your project, make a deliberate **transport choice** (stdio vs HTTP; SSE is legacy), authenticate an OAuth-capable remote server with `claude mcp login`, and verify current ToolSearch loading behavior with `/context`.

Deliverable (toolkit chain 9 of 12)

A working `.mcp.json` in the toolkit repo exposing a `dashboard-metrics` server, plus a written transport decision.

> ***START MARKER:*** *In* `business-dashboard`*, terminal available.*

Fell behind? Any Node project with a git repo works for this lab - run the Lab 2A recovery prompt first.

Phase 1: Build a Small Local Server - stdio (12 min)

Step 1 - start Claude and scaffold a minimal server:

```
Create an MCP server at mcp-server/index.js (plain Node, using @modelcontextprotocol/sdk)
named "dashboard-metrics" with two tools:
1. get_metric(metric_name) - returns mock data for: revenue (48250 USD, up 12%), customers
(342, up 8%), churn (2.1%, down 0.3%), nps (72, up 4 points). Unknown names return an error
listing valid options.
2. get_dashboard_summary() - returns all metrics as JSON.
Use the stdio transport. Install dependencies and verify it starts without errors.
```

Expected output marker: Claude reports the server compiles/starts silently (stdio servers wait on stdin - no output is correct).

Step 2 - register it:

```
Add this server to .mcp.json at the project root:
{
"mcpServers": {
"dashboard-metrics": {
"command": "node",
"args": ["mcp-server/index.js"]
}
}
}
```

Step 3 - reload and verify:

```
/clear
```

```
What MCP tools do you have available? List them.
```

Expected output marker: `get_metric` and `get_dashboard_summary` listed under the dashboard-metrics server.

Step 4 - use it:

```
What's our current revenue? Use the dashboard metrics tool.
```

Expected output marker: Claude calls `get_metric` and reports 48,250 USD, up 12%. Approve the tool call when prompted.

```
Using the dashboard summary tool, add a "Live KPIs" section to the dashboard UI with the
real numbers baked in.
```

That's the power move: MCP data flowing straight into generated code.

Phase 2: Transport Choice (5 min)

Three transports - pick deliberately:

Transport	When	Auth
stdio	Local tools, your machine, child process - what you just built	none needed
HTTP	Remote/shared servers (team services, SaaS)	OAuth with `claude mcp login` when supported; otherwise the approved server auth method
SSE	Legacy - migrate to HTTP	-

Write your decision:

```
For each of these, tell me stdio or HTTP and why: (1) a local SQLite inspector, (2) our
team's shared Jira bridge, (3) a vendor's hosted analytics API.
```

Expected output marker: local child process -> stdio; shared/hosted service -> Streamable HTTP. Authentication is a separate capability check.

Phase 3: Remote Server + claude mcp login (10 min)

Step 1 - add a remote HTTP server. From the shell (`exit` Claude first) - use the instructor-provided URL, or Anthropic's example server if offered:

```
claude mcp add --transport http team-tools <INSTRUCTOR_PROVIDED_URL>
```

Step 2 - authenticate from the shell when the server advertises OAuth support:

```
claude mcp login team-tools
```

Expected output marker: a browser/device auth flow completes and the CLI reports the server authenticated. (`claude mcp logout team-tools` reverses it.)

No remote server available in class? Expected fallback: run both commands anyway and observe the error messages - then remove with `claude mcp remove team-tools`. The command syntax is the takeaway; note "instructor demo" on your checklist.

Step 3 - verify:

```
claude mcp list
```

Expected output marker: both servers listed with transport type and auth status.

Phase 4: ToolSearch and Context Audit (5 min)

Back in Claude (`claude`):

```
/context
```

Current Claude Code enables MCP ToolSearch automatically by default. Initially, only tool names and server instructions load; relevant tool schemas enter context after discovery or use. Note any server configured with `alwaysLoad`, any provider limitation, or any gateway that disables tool references.

```
/doctor
```

Expected output marker: `/context` shows the actual session rather than an assumed per-server tax, and `/doctor` identifies unused servers when applicable. If team-tools is not needed, remove it:

```
claude mcp remove team-tools
```

Commit:

```
Commit .mcp.json and mcp-server/ with "feat: add dashboard-metrics MCP server (stdio)"
```

FINISH MARKER - Success Checklist

- [ ] dashboard-metrics server responds to both tools from Claude
- [ ] `.mcp.json` committed at project root
- [ ] Transport decision written: stdio vs HTTP for 3 scenarios
- [ ] `claude mcp login` executed (or observed in demo)
- [ ] `claude mcp list` shows servers with auth status
- [ ] ToolSearch behavior checked with `/context`; `alwaysLoad` or provider exceptions recorded; unused servers pruned

If Stuck

Problem	Recovery
Tools don't appear	`.mcp.json` must be at project ROOT (not `.claude/`); `/clear` to reload
Server errors on start	`!node mcp-server/index.js` and paste the error to Claude: "Fix this MCP server startup error"
`claude mcp login` hangs	Corporate proxy blocking the auth callback - note it and use the demo fallback
Wrong data returned	Mock data is whatever Claude generated - verify structure (name/value/trend), not exact numbers
Too many tools in context	Confirm ToolSearch is enabled and supported; inspect `alwaysLoad`; disable unused servers

Debrief Questions

1. When is stdio the wrong choice even for a tool you wrote yourself?

2. What does `claude mcp login` give a team that pasting API keys into config files doesn't?
3. What loads before tool use, what can force full schemas up front, and how do you verify the actual session?